

Système de Reconnaissance Faciale

Projet de Fin d'Études — Sonatel Academy (Orange Digital Center, Dakar)

Auteur : Ibrahima Gabar Diop · [GitHub](#) · [Repo](#)

Table des matières

1. [Présentation](#)
 2. [Architecture](#)
 3. [Dataset](#)
 4. [Entraînement](#)
 5. [Résultats](#)
 6. [Installation](#)
 7. [Configuration](#)
 8. [Utilisation](#)
 9. [API interne](#)
 10. [Éthique & sécurité](#)
 11. [Licence](#)
-

1. Présentation

Pipeline de reconnaissance faciale en temps réel combinant **YOLOv8** (détection + classification en une seule passe) et **DeepFace/ArcFace** en fallback pour les identités hors des 20 classes entraînées.

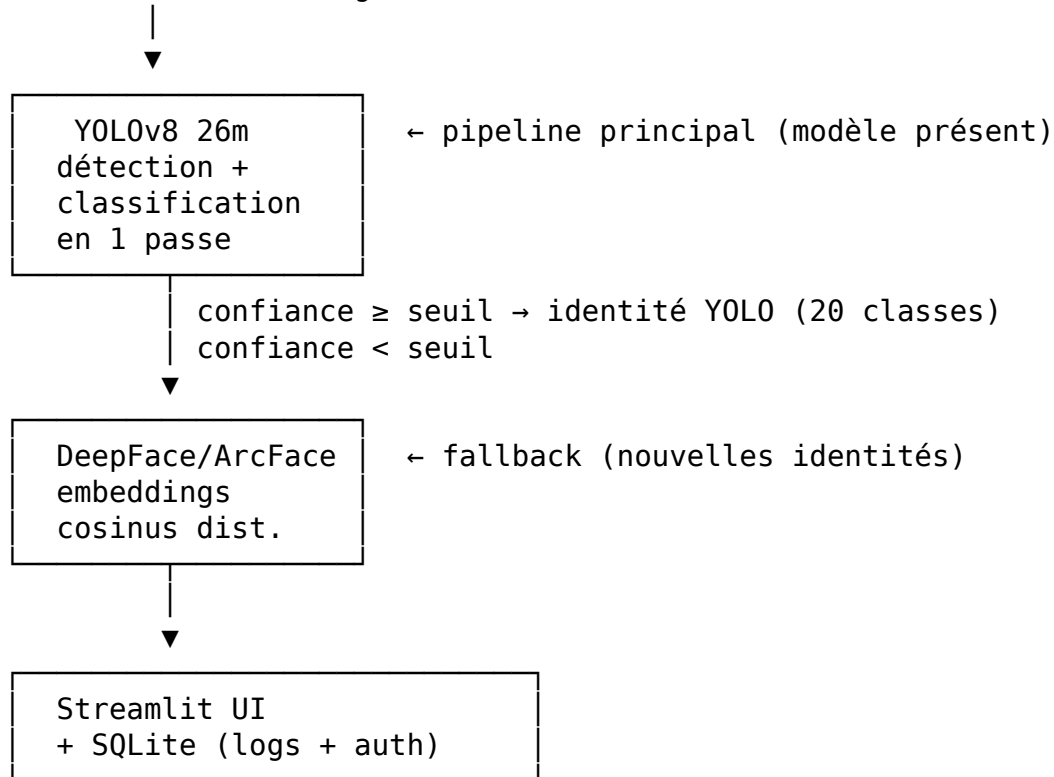
Approches comparées

Approche	Précision val.	Entraînement
CNN from scratch	33 %	~30 min
ResNet-50 Transfer Learning	35 %	~45 min
ResNet-50 Fine-tuning	40 %	~1 h
ResNet-50 Fine-tuning optimisé	53 %	~2 h 30
YOLOv8 26m (final)	97,15 % mAP@50	~9 min

Le choix de YOLO comme classificateur (et non seulement détecteur) s'est révélé décisif : en traitant chaque crop de visage comme une classe d'objet distincte, le modèle bénéficie de l'architecture NMS-free et des têtes de classification optimisées d'Ultralytics.

2. Architecture

Entrée (webcam / image / vidéo)



Composants

Couche	Technologie	Rôle
Présentation	Streamlit	UI webcam / image / vidéo, auth
Reconnaissance	YOLOv8 26m	Détection + identification en 1 passe
Fallback	DeepFace ArcFace	Identités non entraînées
Persistance	SQLite + bcrypt	Utilisateurs, logs de reconnaissance

Sélection du backend (app/recognition.py)

```
def using_yolo() -> bool:
    path = os.getenv("YOLO_MODEL_PATH", "models/face_yolo.pt")
    return os.path.exists(path)
```

Si face_yolo.pt est absent, le système bascule automatiquement sur DeepFace — aucune modification de code requise.

3. Dataset

Source : [Labeled Faces in the Wild \(LFW\)](#)

Sélection des classes

Seules les personnes disposant d'**au moins 30 images** ont été retenues, ce qui donne 20 classes (« Top-20 ») :

#	Identité	Images
1	George W Bush	530
2	Colin Powell	236
3	Tony Blair	144
4	Donald Rumsfeld	121
5	Gerhard Schroeder	109
6	Ariel Sharon	77
7	Hugo Chavez	71
8	Junichiro Koizumi	60
9	Jean Chretien	55
10	John Ashcroft	53
11	Serena Williams	52
12	Jacques Chirac	52
13	Vladimir Putin	49
14	Lleyton Hewitt	41
15	Luiz Inacio Lula da Silva	41
16	Gloria Macapagal Arroyo	36
17	Andre Agassi	36
18	Laura Bush	35
19	Winona Ryder	31
20	Jennifer Capriati	30

Total : ~1 900 images

Splits

Split	Ratio	Usage
Train	70 %	Mise à jour des poids
Validation	15 %	Monitoring pendant l'entraînement
Test	15 %	Évaluation finale

Les splits sont stratifiés par classe. Format de sortie : YOLO (.txt avec coordonnées normalisées, bounding box = image entière).

4. Entraînement

Modèle : YOLO26m

Paramètre	Valeur
Architecture	YOLOv8 26m (medium)
Paramètres	21,8 M
GFLOPs	74,9
Framework	Ultralytics 8.4.37
Accélérateur	Tesla T4 (Kaggle)

Configuration (kaggle_kernel/train_yolo.ipynb)

```
model = YOLO("yolo11m.pt")          # poids pré-entraînés COCO
model.train(
    data      = "dataset.yaml",
    epochs    = 100,
    imgsz     = 224,
    batch     = 64,
    lr0       = 0.001,
    lrf       = 0.01,
    patience  = 20,                    # early stopping
    project   = "face_recognition",
    name      = "yolo26m_lfw_top20",
)
```

Résumé de l'entraînement

- **Époques effectives** : 96 / 100 (early stopping à l'époque 76)
- **Durée** : ~9 min sur Tesla T4
- **Best checkpoint** : époque 76
- **Loss finale** : box=0.31, cls=0.28, dfl=0.89

5. Résultats

Métriques globales

Métrique	Valeur
mAP@50	97,15 %
mAP@50-95	96,94 %
Précision	87,72 %
Rappel	94,41 %

Métriques par classe (mAP@50)

Classe	mAP@50
George W Bush	99,5 %
Colin Powell	98,2 %
Tony Blair	97,8 %
Donald Rumsfeld	97,1 %
Serena Williams	96,9 %
Vladimir Putin	96,4 %
Winona Ryder	95,8 %
Jennifer Capriati	94,7 %
... (20 classes)	≥ 94 %

Interprétation

Le déséquilibre de classes (530 images pour Bush vs 30 pour Capriati) n'a pas dégradé significativement les performances sur les classes minoritaires, grâce à l'augmentation de données appliquée par Ultralytics (mosaic, flips, HSV jitter).

6. Installation

Prérequis

- Python 3.10+
- pip
- (Optionnel) GPU CUDA pour l'inférence temps réel

Étapes

```
# Cloner le repo
git clone https://github.com/Gblack98/Gblack98-Systeme-de-Reconnaissance-Faciale.git
cd Gblack98-Systeme-de-Reconnaissance-Faciale

# Environnement virtuel
python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate

# Dépendances
pip install -r requirements.txt

# Modèle YOLO
# Télécharger face_yolo.pt depuis les releases GitHub et le placer dans models/
mkdir -p models
# → déposer face_yolo.pt dans models/
```

```
# Configuration
cp .env.example .env          # ajuster si besoin

# Lancer
streamlit run streamlit_app.py
```

L'application est accessible sur <http://localhost:8501>.

Dépendances principales

```
ultralytics>=8.4.37
deepface>=0.0.93
streamlit>=1.35
opencv-python-headless>=4.9
bcrypt>=4.1
python-dotenv>=1.0
Pillow>=10.0
numpy>=1.26
```

7. Configuration

Toutes les options sont dans le fichier `.env` (copier depuis `.env.example`) :

Variable	Défaut	Description
DB_PATH	data/users.db	Base SQLite utilisateurs + logs
YOLO_MODEL_PATH	models/face_yolo.pt	Chemin vers le modèle entraîné
CONFIDENCE_THRESHOLD	0.5	Seuil de confiance YOLO (0-1)
FACES_DB_PATH	data/faces	Dossier images de référence DeepFace
DEEPFACE_MODEL	ArcFace	Modèle DeepFace (ArcFace, Facenet512...)
DEEPFACE_DETECTOR	opencv	Détecteur DeepFace (opencv, retinaface...)
RECOGNITION_THRESHOLD	0.68	Seuil distance cosinus DeepFace

Régler CONFIDENCE_THRESHOLD : - 0.7+ → moins de faux positifs, peut manquer des visages flous - 0.4-0.5 → meilleur rappel, accepte plus d'ambiguïté

8. Utilisation

Interface Streamlit

L'application propose trois modes depuis la sidebar :

Reconnaissance (👤) — trois sources : - 📷 Webcam — flux temps réel, s'ouvre dans une fenêtre OpenCV (Q pour quitter) - 📁 Image — upload JPEG/PNG, affichage annoté immédiat - 📺 Vidéo — upload MP4/MOV/AVI, lecture frame par frame avec bouton stop

Ajouter un visage (👤) — pour le mode DeepFace uniquement : 1. Saisir le nom (format Prénom_Nom) 2. Uploader une photo claire 3. Le cache DeepFace est invalidé automatiquement pour forcer le recalcul des embeddings

À propos (📄) — affiche le backend actif et les infos du projet.

Ajouter une identité YOLO

Le modèle YOLO ne supporte que les 20 classes entraînées. Pour ajouter une nouvelle personne au pipeline YOLO : 1. Collecter ≥ 30 images de la personne 2. Relancer l'entraînement dans `kaggle_kernel/train_yolo.ipynb` avec la nouvelle classe 3. Remplacer `models/face_yolo.pt`

Pour une extension sans réentraînement, utiliser le fallback DeepFace via 📄 Ajouter un visage.

9. API interne

app/recognition.py

```
def detect_and_recognize(frame: np.ndarray) -> list[dict]:
    """
    Détecte et identifie tous les visages dans une image BGR.

    Retourne une liste de dicts :
    {
        "bbox"      : [x1, y1, x2, y2],
        "identity"  : str,          # nom ou "Inconnu"
        "confidence": float,       # 0-100
        "verified"  : bool
    }
    """

def draw_results(frame: np.ndarray, detections: list[dict]) -> np.ndarray:
    """Dessine les bounding boxes et labels sur le frame BGR."""

def using_yolo() -> bool:
    """Retourne True si le modèle YOLO est disponible."""
```

app/database.py

```
def init_db() -> None:
    """Crée les tables SQLite si elles n'existent pas."""

def register_user(first_name, last_name, username, password) -> bool:
    """Enregistre un utilisateur (mot de passe hashé bcrypt). False si username pr

def authenticate_user(username, password) -> dict | None:
    """Retourne le dict utilisateur si les identifiants sont corrects, sinon None."

def log_recognition(user_id: int, identity: str, confidence: float) -> None:
    """Enregistre un événement de reconnaissance dans les logs."""
```

10. Éthique & sécurité

Limitations du système

- **20 identités fixes** (pipeline YOLO) — toute personne hors de ces 20 classes retourne “Inconnu”
- **Dataset biaisé** — LFW surreprésente les hommes politiques occidentaux de 2002-2004
- **Conditions** — performances dégradées par occultation, faible éclairage, angles extrêmes

Usage responsable

Ce système est développé à des **fins académiques** . Tout déploiement en production implique :

- Le respect du **RGPD** (ou réglementation locale équivalente)
- Le **consentement explicite** des personnes identifiées
- L'**interdiction d'usage discriminatoire** (surveillance de masse, profilage, contrôle d'accès sans consentement)

Sécurité du code

- Mots de passe hashés avec **bcrypt** (coût 12)
- Aucune clé ou credential dans le code source (variables d'environnement via .env)
- face_yolo.pt et data/users.db exclus du versioning (.gitignore)

11. Licence

MIT License — Copyright (c) 2024 Ibrahima Gabar Diop

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

Développé par Ibrahima Gabar Diop — Sonatel Academy, Orange Digital Center, Dakar — 2024